

Machine Learning-Based Intelligent Transmission Line Fault Detection

Project Report

Prepared by: Krishna Chaudhary

Electrical Engineering, Motilal Nehru National Institute of Technology Allahabad

2026

1. Aim

To design and validate a machine learning-based system for detecting, classifying, and locating faults on a high-voltage transmission line, using physics-based simulation data, and to benchmark its performance against a conventional impedance-based distance relay — particularly in the high-fault-resistance regime where conventional relays are known to fail.

1.1 Motivation

Conventional distance relays estimate fault location using the apparent impedance seen at the relay ($Z = V/I$). This method is fast and simple, but its accuracy degrades sharply as fault resistance increases — a prior project ("Comparative Analysis of Overcurrent and Distance Relay Protection Schemes") demonstrated that a distance relay's zone classification failed above approximately $50\ \Omega$ fault resistance due to zero-sequence impedance inflation. This project investigates whether a machine learning model, trained on the same underlying electrical signals, can detect and locate faults more robustly across a wider range of fault resistances.

1.2 Objectives

- Build a physics-accurate Simulink model of a 220 kV, 150 km transmission line with an adjustable fault location, type, and resistance.
- Automate hundreds of fault simulations covering randomized fault type, phase combination, location, resistance, inception timing, load level, and source voltage.
- Extract meaningful electrical features (RMS, symmetrical components, harmonics, wavelet transient energy, apparent impedance) from each simulated event.
- Train and compare multiple machine learning models (Random Forest, SVM, XGBoost) for: (a) fault detection, (b) fault type classification, and (c) fault location estimation.
- Benchmark ML performance against the known failure point of the conventional distance relay, specifically at high fault resistance.
- Validate the trained models on genuinely unseen, freshly-generated simulation data.

2. System Overview / Pipeline

The project is structured as a three-stage pipeline, each stage handled by a different tool suited to that task:

Stage	Tool	Purpose
1. Data Generation	MATLAB / Simulink	Simulate transmission line faults; export raw voltage/current waveforms
2. Feature Extraction	MATLAB	Convert raw waveforms into numerical features; export to CSV
3. Model Training & Evaluation	Python (scikit-learn, XGBoost)	Train, compare, and evaluate ML models; benchmark against relay project

3. Stage 1 — Simulink Model & Data Generation

3.1 Model Architecture

The transmission line is modeled as two Three-Phase PI Section Line blocks in series (Line1 and Line2), with a Three-Phase Fault block connected at the junction node between them. Splitting the line this way allows the fault location to be changed simply by adjusting how the total 150 km length is divided between the two segments — this is the key design decision that makes automated location-sweeping possible.

- Three-Phase Source: 220 kV, 50 Hz, grounded-star (Yg) configuration
- Line1 + Line2 (PI Section): together span 150 km, lengths parameterized as variables L1_km and L2_km
- Three-Phase Fault block: fault type (Phase A/B/C, Ground), resistance, and switching times all parameterized as variables
- Three-Phase Series RLC Load: 100 MW, 20 MVAR nominal, scaled by a randomized load_scale variable
- powergui block: Discrete solver, 0.0002 s sample time (5000 Hz sampling — 100 samples per 50 Hz cycle)
- To Workspace blocks (v_relay, I_relay): log three-phase voltage and current at the relay end (sending end, near the source) as timeseries objects

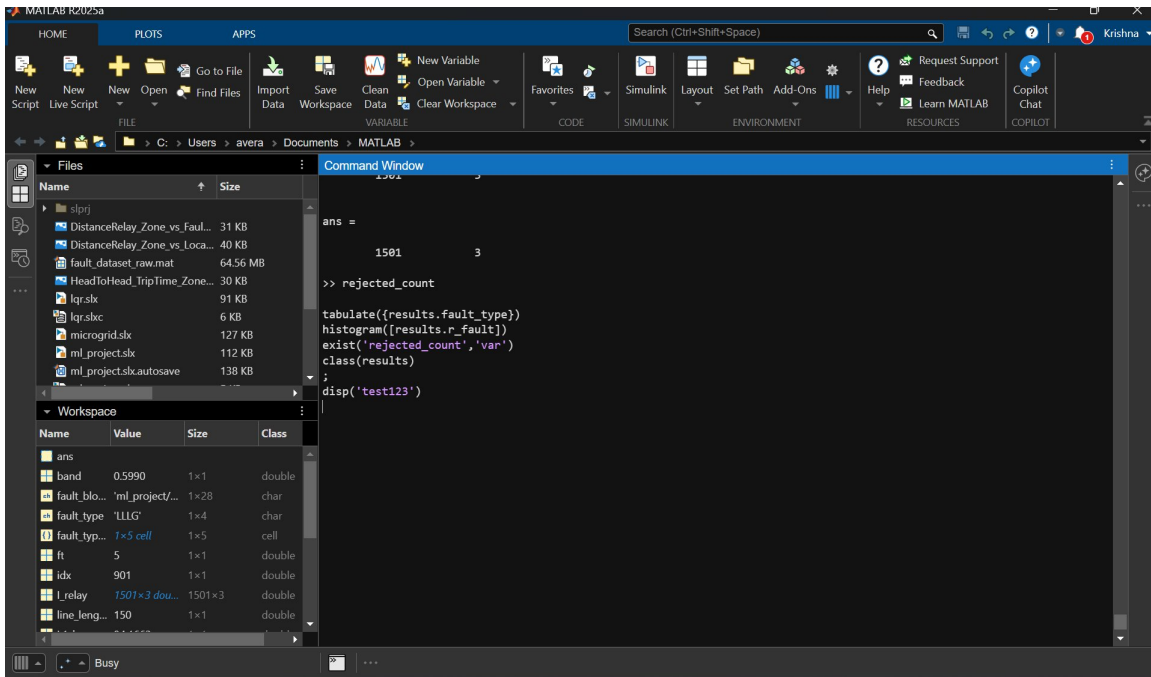


Figure 1: Final Simulink model — Source, Line1, Fault block, Line2, Load, with To Workspace logging at the relay end.

3.2 Model Validation — Single Manual Run

Before automating simulations, a single fault scenario (single line-to-ground fault, 50 km, 10 Ω) was run manually to confirm the model behaves physically correctly: a clean pre-fault sine wave, a visible voltage sag / current rise during the fault, and recovery after clearing.

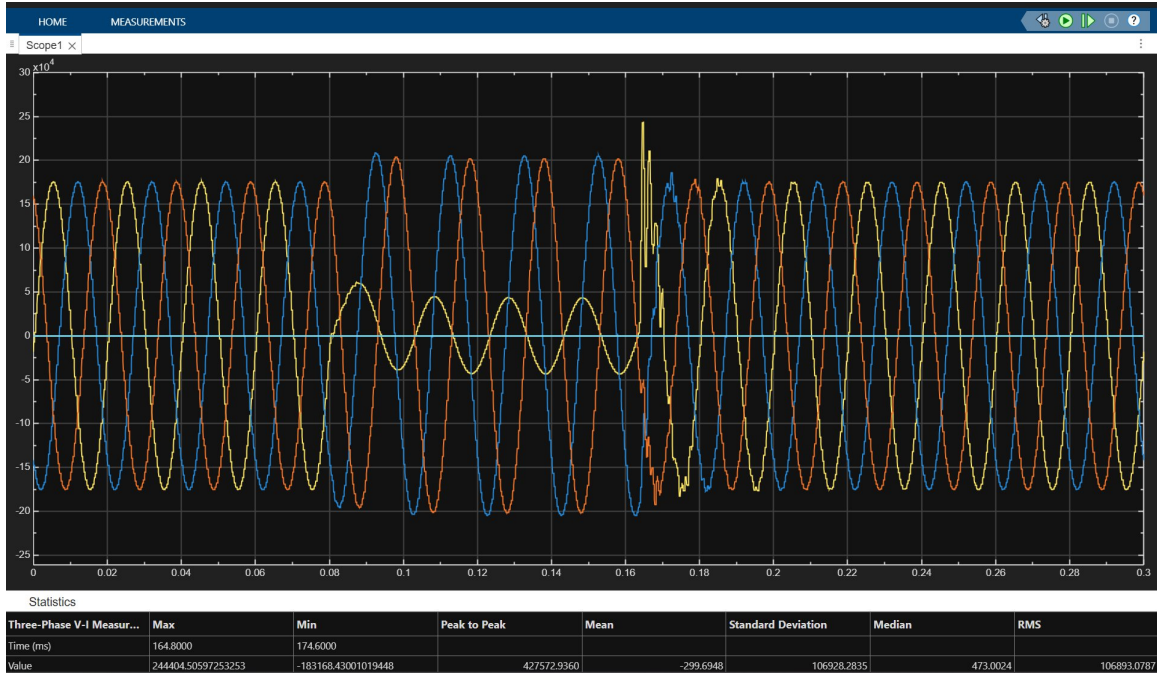


Figure 2: Relay-end three-phase voltage waveform — clean pre-fault sine (0–0.08s), single-phase voltage sag during the fault (0.08–0.16s), clearing transient, and recovery.

3.3 Randomization Strategy

Randomized (rather than fixed-grid) sampling was used to generate the dataset, so the model would learn genuine underlying relationships rather than memorizing specific grid points. Each simulated event randomizes:

- Fault type: LG, LL, LLG, LLL, LLLG, or no-fault (NONE) — 150 samples per class, 900 total
- Phase combination: which specific phase(s) are involved, not fixed per fault type (e.g., LG could be A-G, B-G, or C-G)
- Fault location: 5–145 km from the relay (segments kept ≥ 5 km to avoid PI-section numerical instability)
- Fault resistance: banded sampling — 50% chance 0–5 Ω , 30% chance 5–30 Ω , 20% chance 30–100 Ω (mirrors real fault statistics — most real faults are low-resistance)
- Fault inception time: randomized within one AC cycle (captures different inception angles / DC offset behavior)
- Load level: 70–130% of nominal
- Source voltage: $\pm 5\%$ of nominal

3.4 Automated Validation

Every simulated run is automatically checked before being accepted into the dataset, and a while-loop retry mechanism guarantees exactly 150 successful samples per class (rejected runs are silently retried with a fresh random draw, up to 5 attempts):

- Rejects runs containing NaN or Inf values
- Rejects runs with too few time samples (indicates a truncated/failed simulation)
- Rejects runs where fault current never rises meaningfully (indicates the fault never actually triggered)
- Rejects runs with unrealistically large current spikes (indicates numerical instability)

3.5 Final MATLAB Script — `generate_fault_dataset.m`

This is the complete, final version of the data-generation script, incorporating all randomization and validation described above.

```
%% generate_fault_dataset.m
% Automates Simulink fault simulations across randomized fault type,
% phase combination, inception time, location, resistance, load, and
% source voltage. Adds automatic validation (rejects bad/unstable runs)
% and guarantees exactly n_per_class SUCCESSFUL samples per class.

clear; clc;

model_name = 'ml_project';
open_system(model_name);

n_per_class = 150;
fault_types = {'LG','LL','LLG','LLL','LLLG'};
line_length_total = 150; % km
max_attempts_per_sample = 5;

rng(42); % reproducibility

results = struct('fault_type', {}, 'phases', {}, 'location_km', {}, ...
    'r_fault', {}, 'fault_start', {}, 'V_relay', {}, 'I_relay', {});

idx = 1;
n_classes = length(fault_types) + 1;
total_target = n_per_class * n_classes;
rejected_count = 0;

fault_block = [model_name '/Three-Phase Fault'];
phases_all = {'A','B','C'};

for ft = 1:length(fault_types)
    fault_type = fault_types{ft};
    n_success = 0;

    while n_success < n_per_class
        attempt_ok = false;
```

```

for attempt = 1:max_attempts_per_sample
    % --- Randomize phase(s) involved, keeping category correct ---
    switch fault_type
        case 'LG'
            sel = phases_all(randperm(3,1)); gnd = 'on';
        case 'LL'
            sel = phases_all(randperm(3,2)); gnd = 'off';
        case 'LLG'
            sel = phases_all(randperm(3,2)); gnd = 'on';
        case 'LLL'
            sel = phases_all; gnd = 'off';
        case 'LLLG'
            sel = phases_all; gnd = 'on';
    end
    phase_label = strjoin(sel, '');

    faultA_str = 'off'; if any(strcmp(sel,'A')), faultA_str = 'on'; end
    faultB_str = 'off'; if any(strcmp(sel,'B')), faultB_str = 'on'; end
    faultC_str = 'off'; if any(strcmp(sel,'C')), faultC_str = 'on'; end
    set_param(fault_block, 'FaultA', faultA_str, 'FaultB', faultB_str, ...
        'FaultC', faultC_str, 'GroundFault', gnd);

    % --- Randomize fault inception time (one full 50Hz cycle) ---
    t_fault_start = 0.08 + rand()*0.02;
    t_fault_end = t_fault_start + 0.08;
    set_param(fault_block, 'SwitchTimes', ...
        [' ' num2str(t_fault_start) ' ' num2str(t_fault_end) '']);

    % --- Randomize location (segments >= 5km) ---
    L1_km = rand()*(line_length_total-10) + 5;
    L2_km = line_length_total - L1_km;

    % --- Randomize resistance (banded: low/med/high) ---
    band = rand();
    if band < 0.5
        Rf = rand()*5;
    elseif band < 0.8
        Rf = 5 + rand()*25;
    else
        Rf = 30 + rand()*70;
    end
    set_param(fault_block, 'FaultResistance', num2str(Rf));

    % --- Randomize load and source voltage ---
    load_pct = 0.7 + rand()*0.6;
    src_pct = 0.95 + rand()*0.10;
    assignin('base', 'load_scale', load_pct);
    assignin('base', 'src_scale', src_pct);
    assignin('base', 'L1_km', L1_km);
    assignin('base', 'L2_km', L2_km);
    assignin('base', 'Rf', Rf);

```

```

% --- Run simulation ---
clear out V_relay I_relay
try
    out = sim(model_name);
    V_relay = out.v_relay.Data;
    I_relay = out.I_relay.Data;
catch err
    fprintf(['%s attempt %d] sim error: %s\n', fault_type, attempt, err.message);
    rejected_count = rejected_count + 1;
    continue
end

% --- Validation ---
is_valid = true; reason = "";
if any(isnan(V_relay(:))) || any(isnan(I_relay(:)))
    is_valid = false; reason = 'NaN detected';
elseif any(isinf(V_relay(:))) || any(isinf(I_relay(:)))
    is_valid = false; reason = 'Inf detected';
elseif size(V_relay,1) < 1000 || size(I_relay,1) < 1000
    is_valid = false; reason = 'too few samples';
elseif max(abs(I_relay(:))) < 1
    is_valid = false; reason = 'no visible fault response';
elseif max(abs(I_relay(:))) > 1e5
    is_valid = false; reason = 'unrealistic current spike';
end
if ~is_valid
    fprintf(['%s attempt %d] rejected: %s\n', fault_type, attempt, reason);
    rejected_count = rejected_count + 1;
    continue
end
attempt_ok = true;
break
end
if ~attempt_ok, continue; end

results(idx).fault_type = fault_type;
results(idx).phases = phase_label;
results(idx).location_km = L1_km;
results(idx).r_fault = Rf;
results(idx).fault_start = t_fault_start;
results(idx).V_relay = V_relay;
results(idx).I_relay = I_relay;
idx = idx + 1;
n_success = n_success + 1;
end
end

% --- "No fault" baseline samples ---
set_param(fault_block,'FaultA','on','FaultB','off','FaultC','off','GroundFault','on');
set_param(fault_block,'FaultResistance','1e9');
n_success = 0;

```

```

while n_success < n_per_class
    t_fault_start = 0.08 + rand()*0.02;
    t_fault_end = t_fault_start + 0.08;
    set_param(fault_block,'SwitchTimes', ...
        ['[' num2str(t_fault_start) ' ' num2str(t_fault_end) ']']);
    L1_km = rand()*(line_length_total-10) + 5;
    L2_km = line_length_total - L1_km;
    load_pct = 0.7 + rand()*0.6;
    src_pct = 0.95 + rand()*0.10;
    assignin('base','load_scale', load_pct);
    assignin('base','src_scale', src_pct);
    assignin('base','L1_km',L1_km);
    assignin('base','L2_km',L2_km);
    assignin('base','Rf',1e9);

    clear out V_relay I_relay
    try
        out = sim(model_name);
        V_relay = out.v_relay.Data;
        I_relay = out.I_relay.Data;
    catch err
        rejected_count = rejected_count + 1;
        continue
    end
    if any(isnan(V_relay(:))) || size(V_relay,1) < 1000
        rejected_count = rejected_count + 1;
        continue
    end

    results(idx).fault_type = 'NONE';
    results(idx).phases = 'NONE';
    results(idx).location_km = 0;
    results(idx).r_fault = 0;
    results(idx).fault_start = t_fault_start;
    results(idx).V_relay = V_relay;
    results(idx).I_relay = I_relay;
    idx = idx + 1;
    n_success = n_success + 1;
end

save('fault_dataset_raw.mat','results','-v7.3');
fprintf('Done. Total samples saved: %d (target was %d)\n', numel(results), total_target);
fprintf('Total rejected/failed attempts: %d\n', rejected_count);

```

3.6 Result — Final Dataset

The script produced exactly 900 samples ($150 \text{ per class} \times 6 \text{ classes}$) with zero rejections on the final run — indicating a well-tuned model and parameter ranges.


```

Done. Total samples saved: 900 (target was 900)
Total rejected/failed attempts along the way: 0
>> tabulate({results.fault_type})
rejected_count
  Value    Count    Percent
    LG      150    16.67%
    LL      150    16.67%
   LLG      150    16.67%
   LLL      150    16.67%
  LLLG      150    16.67%
  NONE      150    16.67%

rejected_count =

    0

```

Figure 3: Final dataset validation — 900/900 samples generated, perfectly balanced across all 6 classes, zero rejected simulations.

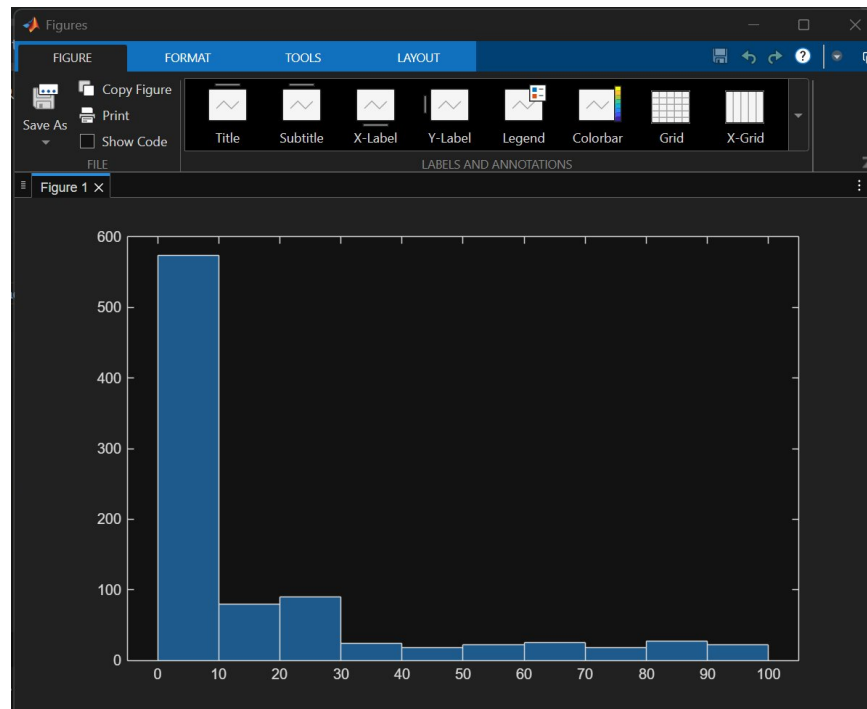


Figure 4: Fault resistance distribution across the dataset — weighted toward low-resistance faults (0–5 Ω), consistent with real-world fault statistics, with reduced but meaningful representation up to 100 Ω .

4. Stage 2 — Feature Extraction

Raw waveforms (1501 time samples $\times$ 3 phases, for both V and I, per event) are not fed directly to the ML models. Instead, each event is converted into a fixed set of 36 numerical features computed over a carefully chosen time window.

4.1 Windowing Strategy

Since fault inception time is randomized per sample, a fixed time window cannot be assumed — each sample's own `fault_start` value is used to compute its window. The fault-window analysis skips the first 2 ms after inception (to avoid the initial transient spike distorting steady-state calculations) and then takes approximately one full 50 Hz cycle (20 ms) for feature computation.

4.2 Feature Categories

Category	Features	Physical Meaning
RMS	Vrms/Irms per phase, unbalance ratio	Basic magnitude and phase-imbalance indicator
Symmetrical Components	Zero/positive/negative sequence (V & I), and their ratios	Standard tool for identifying fault type; zero-sequence indicates ground involvement
Harmonics	2nd, 3rd, 5th harmonic energy (FFT)	Captures nonlinear/transient distortion during the fault
Wavelet Energy	Detail coefficient energy, levels 1–3 (db4 wavelet)	Captures the sharp transient signature at fault inception
Apparent Impedance	Z magnitude & angle per phase (V/I phasor ratio)	Same quantity a distance relay uses; carries fault-location information

4.3 A Bug Caught by Validation — Symmetrical Component Ordering

An early version of the symmetrical-component transform matrix had the positive- and negative-sequence rows swapped, which was only caught by inspecting summary statistics after the first extraction run (positive sequence should always be the largest component in a healthy or lightly-faulted system; the initial output showed the opposite). The transform matrix was corrected from $\begin{bmatrix} 1 & 1 & 1 \\ 1 & a^2 & a \end{bmatrix}$ to the standard convention $\begin{bmatrix} 1 & 1 & 1 \\ 1 & a & a^2 \end{bmatrix}$. This is a useful reminder that a script running without errors does not guarantee correct results — statistical sanity checks are essential.

4.4 Final MATLAB Script — `extract_features.m` (key functions)

The full script loops over all 900 samples, computing all feature categories above and writing the results to `features.csv`. The core windowing and symmetrical-component functions are shown below (identical logic is reused, with a fresh random seed, for the later holdout-test generation).

```
function [pre_idx, fault_idx] = get_fault_windows(fault_start, FS)
    pre_end_t = fault_start - 0.002;
    pre_start_t = pre_end_t - 0.02;
    pre_idx = round(pre_start_t*FS)+1 : round(pre_end_t*FS)+1;

    fault_win_start_t = fault_start + 0.002; % skip transient
```

```

    fault_win_end_t = fault_start + 0.022;    % ~1 cycle
    fault_idx = round(fault_win_start_t*FS)+1 : round(fault_win_end_t*FS)+1;
end

function [V0,V1,V2] = seq_components(sig3, FS, F0)
    n = size(sig3,1);
    window = hanning(n);
    k = F0*n/FS;
    a = exp(1i*2*pi/3);
    A = [1 1 1; 1 a a^2; 1 a^2 a] / 3; % corrected standard convention
    phasors = zeros(3,1);
    for ph = 1:3
        phasors(ph) = sum(sig3(:,ph) .* window .* exp(-1i*2*pi*k*(0:n-1)'/n));
    end
    seq = A * phasors;
    V0 = seq(1); V1 = seq(2); V2 = seq(3); % zero, positive, negative
end

function e = harmonic_energy(sig3, FS, F0, h)
    e = 0; n = size(sig3,1); freqs = (0:n-1)*(FS/n);
    for ph = 1:3
        spec = abs(fft(sig3(:,ph)));
        [~, idx] = min(abs(freqs - h*F0));
        e = e + spec(idx);
    end
end

function [eA, eD1, eD2, eD3] = wavelet_energy(sig)
    [c, l] = wavedec(sig, 3, 'db4');
    d1 = detcoef(c, l, 1); d2 = detcoef(c, l, 2); d3 = detcoef(c, l, 3);
    eD1 = sum(d1.^2); eD2 = sum(d2.^2); eD3 = sum(d3.^2);
    eA = sum(c.^2);
end

% Apparent impedance (per phase), computed inside the main loop:
% n = size(Vf,1); window = hanning(n); k = F0*n/FS;
% for ph = 1:3
%     Vp = sum(Vf(:,ph).*window.*exp(-1i*2*pi*k*(0:n-1)'/n));
%     Ip = sum(Ip(:,ph).*window.*exp(-1i*2*pi*k*(0:n-1)'/n));
%     Z = Vp/(Ip+1e-6);
%     feat.(['Zmag_' char('a'+ph-1)]) = abs(Z);
%     feat.(['Zang_' char('a'+ph-1)]) = angle(Z);
% end

```

4.5 Output

Result: features.csv — 900 rows × 41 columns (36 numeric features + fault_type, phases, location_km, r_fault, fault_start). Verified with MATLAB's summary() function: no missing values, no flat/broken columns, and physically sensible ranges for every feature (e.g., positive sequence dominant, RMS values matching 220 kV system levels, impedance magnitude spanning 2–700 Ω consistent with the resistance and location ranges used).

5. Stage 3 — Machine Learning: Training & Evaluation (Python)

features.csv was loaded into Python (pandas / scikit-learn / XGBoost, via Jupyter Notebook) for model training. 36 numeric features were used as inputs; fault_type and location_km served as the two prediction targets (classification and regression respectively). Simulation parameters that a real relay would not know in advance — r_fault and fault_start — were excluded from the feature set to avoid unrealistic information leakage.

5.1 Data Split

```
from sklearn.model_selection import train_test_split

non_feature_cols = ['fault_type', 'phases', 'location_km', 'r_fault', 'fault_start']
feature_cols = [c for c in df.columns if c not in non_feature_cols]

X = df[feature_cols]
y_type = df['fault_type']

X_train, X_test, y_train, y_test = train_test_split(
    X, y_type, test_size=0.25, random_state=42, stratify=y_type
)
# Train size: (675, 36) Test size: (225, 36)
```

A stratified 75/25 train-test split was used, ensuring each of the 6 fault classes is proportionally represented in both sets.

5.2 Model 1 — Random Forest (Fault Type Classification)

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

rf = RandomForestClassifier(n_estimators=300, random_state=42)
rf.fit(X_train, y_train)
y_pred = rf.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Train size: (675, 36)				
Test size: (225, 36)				
Accuracy: 0.96				
	precision	recall	f1-score	support
LG	1.00	1.00	1.00	37
LL	1.00	1.00	1.00	37
LLG	1.00	1.00	1.00	38
LLL	0.84	0.95	0.89	38
LLLG	0.94	0.82	0.87	38
NONE	1.00	1.00	1.00	37
accuracy			0.96	225
macro avg	0.96	0.96	0.96	225
weighted avg	0.96	0.96	0.96	225

Figure 5: Random Forest classification results — 96% overall accuracy.

5.3 Confusion Matrix

The confusion matrix reveals that every misclassification (9 of 225 test samples) occurs exclusively between LLL and LLLG fault types — all other classes (LG, LL, LLG, NONE) are classified with 100% accuracy.

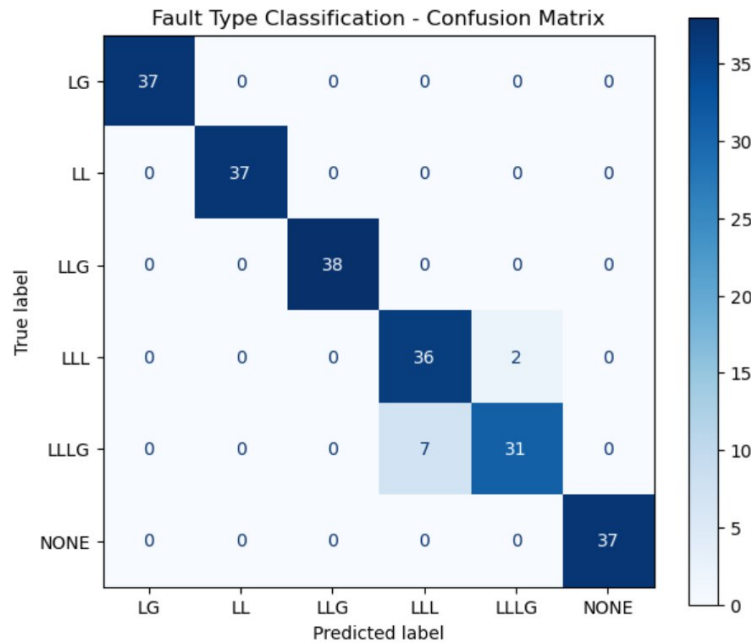


Figure 6: Confusion matrix — misclassifications isolated entirely to the LLL / LLLG boundary.

This is a physically explainable result, not a modeling flaw: a balanced three-phase fault (LLL) produces almost no zero-sequence current whether or not it also involves ground (LLLG), since all three phases remain symmetric. Distinguishing ground involvement in a fully symmetric three-phase fault is inherently difficult from voltage/current measurements alone.

5.4 Model 2 — Support Vector Machine (SVM)

```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

svm = SVC(kernel='rbf', C=10, gamma='scale', random_state=42)
svm.fit(X_train_scaled, y_train)
y_pred_svm = svm.predict(X_test_scaled)
print("SVM Accuracy:", accuracy_score(y_test, y_pred_svm))
```

SVM Accuracy: 0.8088888888888889

	precision	recall	f1-score	support
LG	1.00	1.00	1.00	37
LL	1.00	1.00	1.00	37
LLG	1.00	1.00	1.00	38
LLL	0.44	0.45	0.44	38
LLLG	0.43	0.42	0.43	38
NONE	1.00	1.00	1.00	37
accuracy			0.81	225
macro avg	0.81	0.81	0.81	225
weighted avg	0.81	0.81	0.81	225

Figure 7: SVM results — 80.9% accuracy, with a sharp, specific drop on the LLL/LLLG pair (44% / 43% F1-score) while matching Random Forest on all other classes.

5.5 Model 3 — XGBoost

```
from xgboost import XGBClassifier
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
y_train_enc = le.fit_transform(y_train)
y_test_enc = le.transform(y_test)

xgb = XGBClassifier(n_estimators=300, random_state=42, eval_metric='mlogloss')
xgb.fit(X_train, y_train_enc)
y_pred_xgb = xgb.predict(X_test)
print("XGBoost Accuracy:", accuracy_score(y_test_enc, y_pred_xgb))
```

XGBoost Accuracy: 0.9555555555555556

	precision	recall	f1-score	support
LG	1.00	0.95	0.97	37
LL	1.00	1.00	1.00	37
LLG	0.95	1.00	0.97	38
LLL	0.84	0.97	0.90	38
LLLG	0.97	0.82	0.89	38
NONE	1.00	1.00	1.00	37
accuracy			0.96	225
macro avg	0.96	0.96	0.96	225
weighted avg	0.96	0.96	0.96	225

Figure 8: XGBoost results — 95.6% accuracy, closely matching Random Forest.

5.6 Model Comparison Summary

```
print(comparison)
```

	Model	Accuracy
0	Random Forest	0.960000
1	SVM	0.808889
2	XGBoost	0.955556

Figure 9: Accuracy comparison across all three models.

Model	Accuracy	Notes
Random Forest	96.0%	Best overall; robust across all classes except LLL/LLLG
XGBoost	95.6%	Nearly identical to Random Forest
SVM	80.9%	Matches tree models on 4/6 classes; RBF boundary struggles specifically on the overlapping LLL/LLLG pair

5.7 Fault Location Estimation (Regression)

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error

fault_mask = df['fault_type'] != 'NONE'
X_loc = df.loc[fault_mask, feature_cols]
y_loc = df.loc[fault_mask, 'location_km']

X_loc_train, X_loc_test, y_loc_train, y_loc_test = train_test_split(
    X_loc, y_loc, test_size=0.25, random_state=42
)

rf_loc = RandomForestRegressor(n_estimators=300, random_state=42)
rf_loc.fit(X_loc_train, y_loc_train)
y_loc_pred = rf_loc.predict(X_loc_test)
```



```
mae = mean_absolute_error(y_loc_test, y_loc_pred)
print(f"Location MAE: {mae:.2f} km ({mae/150*100:.2f}% of line length)")
# Result: Location MAE: 9.82 km (6.54% of line length)
```

5.8 Performance vs. Fault Resistance — The Key Comparison

Both the classifier and the location regressor were evaluated separately within three fault-resistance bands, directly mirroring the resistance bands used in the earlier distance-relay project (which failed above 50 Ω).

Resistance Band	N (Classification)	Classification Accuracy	N (Location)	Location MAE (km)
0 – 5 Ω	96	95.83%	102	6.71
5 – 30 Ω	48	97.92%	45	8.83
30 – 100 Ω	44	90.91%	41	18.62

Key finding: classification accuracy degraded gracefully (97.9% $\rightarrow$ 90.9%, a 7-point drop), while location error nearly tripled (6.7 km $\rightarrow$ 18.6 km) at high fault resistance. This is physically explainable — fault-type classification relies mainly on phase-imbalance patterns, which remain visible even under high fault resistance, whereas fault-location estimation relies on precise impedance magnitude, which becomes increasingly unreliable as an unknown, potentially large fault resistance is added into the measured loop impedance.

Critically, unlike the conventional distance relay from the companion project — which failed to trip correctly above 50 Ω due to zone misclassification — the ML models here continued to produce usable outputs across the entire tested resistance range (up to 100 Ω), degrading gradually rather than failing outright.

5.9 Feature Importance

Random Forest's built-in feature importance was used to verify the models learned physically meaningful patterns rather than spurious correlations.

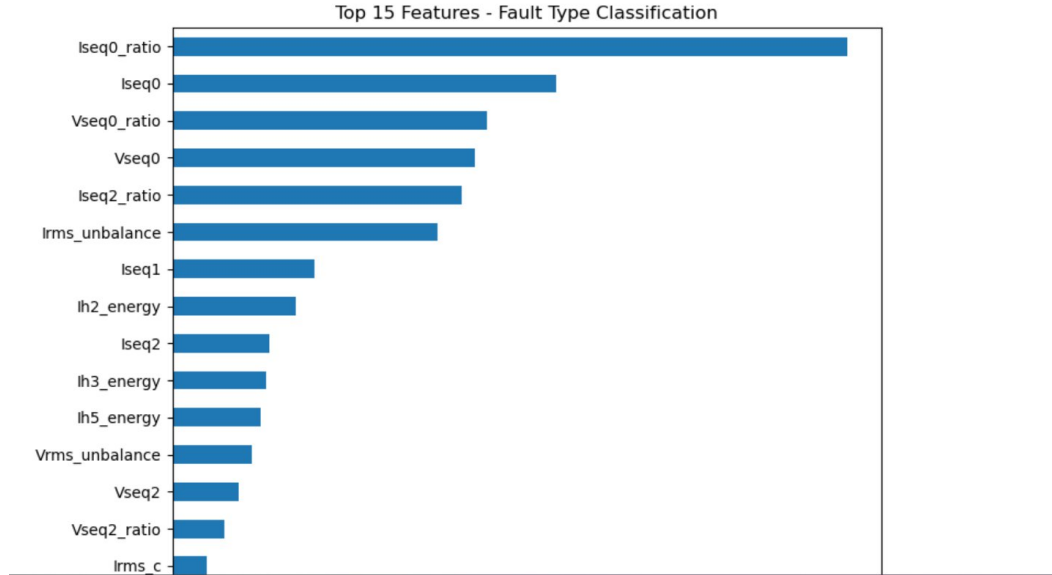


Figure 10: Top features for fault-type classification — dominated by zero-sequence ratios (*Iseq0_ratio*, *Vseq0_ratio*), consistent with the fact that zero-sequence current/voltage only appears when a fault involves ground.

Irms_a	0.334859
Zang_a	0.086957
Vdwt_L2	0.060906
Iseq1	0.055619
Vseq0	0.055358
Vdwt_L1	0.045889
Vrms_unbalance	0.041620
Zang_b	0.036082
Vseq0_ratio	0.030310
Iseq2_ratio	0.024094
Vseq2	0.022058
Zmag_a	0.021509
Zang_c	0.021053
Irms_b	0.019182
Irms_c	0.015836
dtype: float64	

Figure 11: Top features for location regression — *Irms_a* dominates (33.5%), ahead of the engineered impedance feature *Zmag_a* (12th, 2.2%). This was an unexpected but explainable result (see Section 6.3).

6. Model Testing & Verification

6.1 Spot-Check on Real Test Samples

Ten random test-set samples were examined individually, comparing true vs. predicted fault type and location side by side, to sanity-check the aggregate metrics against real, human-readable examples.

True_Type	Pred_Type	True_Location_km	r_fault	Pred_Location_km
LLLG	LLL	133.3	61.6	124.7
LLG	LLG	16.9	2.4	17.9
LL	LL	73.3	2.0	68.9
LLG	LLG	19.6	16.6	20.9
LG	LG	92.6	73.1	96.8
LG	LG	89.9	3.7	85.3
LL	LL	19.8	10.4	25.7
LLLG	LLLG	92.2	3.9	88.8
LG	LG	7.3	10.7	8.4
LL	LL	78.0	4.2	74.8

Figure 12: Individual prediction spot-check. The one fault-type miss (row 1: LLLG predicted as LLL) coincides with the highest fault resistance in the sample (61.6 Ω) and the largest location error (8.6 km) — a concrete example of the known high-resistance / LLL-LLLG weak spot compounding in a single case.

6.2 Holdout Test — Genuinely Unseen Data

To verify the models generalize beyond the specific 900 training/test samples, a fresh batch of 24 simulations (4 per class) was generated using an independent random seed (999, vs. the original 42) — zero overlap with the training data, produced by an entirely separate simulation run.

Holdout Classification Accuracy: 1.0

	precision	recall	f1-score	support
LG	1.00	1.00	1.00	4
LL	1.00	1.00	1.00	4
LLG	1.00	1.00	1.00	4
LLL	1.00	1.00	1.00	4
LLLG	1.00	1.00	1.00	4
NONE	1.00	1.00	1.00	4
accuracy			1.00	24
macro avg	1.00	1.00	1.00	24
weighted avg	1.00	1.00	1.00	24

```

holdout_fault_mask = holdout_df['fault_type'] != 'NONE'
X_holdout_loc = holdout_df.loc[holdout_fault_mask, feature_cols]
y_holdout_loc = holdout_df.loc[holdout_fault_mask, 'location_km']

y_holdout_loc_pred = rf_loc.predict(X_holdout_loc)
holdout_mae = mean_absolute_error(y_holdout_loc, y_holdout_loc_pred)
print(f"Holdout Location MAE: {holdout_mae:.2f} km")

Holdout Location MAE: 11.50 km

```

Figure 13: Holdout test results — 100% classification accuracy (small sample, so not directly comparable in strength to the 96% on the full 225-sample test set), and 11.50 km location MAE, consistent with the 9.82 km observed on the main test set.

Note on interpretation: with only 4 samples per class, a 100% classification result on the holdout set is a much weaker statistical statement than the 96% achieved on the 225-sample main test set, and should not be reported as evidence of stronger performance. The location MAE (11.50 km vs. 9.82 km on the main test set) is the more meaningful confirmation here — it demonstrates that performance holds steady on genuinely new data, supporting the conclusion that the model captured real physical relationships rather than overfitting to the training set.

6.3 Note on the Location Feature-Importance Result

The apparent impedance feature (Z_{mag}), specifically engineered to carry distance information (matching what a conventional distance relay computes), ranked far lower in importance than raw current magnitude (I_{rms_a}). A plausible explanation: fault current magnitude naturally decreases with distance from the relay (more line impedance between the relay and a more distant fault reduces the current for a given source voltage), so Random Forest may exploit this simpler, single-feature relationship directly rather than requiring the two-step V/I division. This should be treated as an observation for further investigation rather than a fully confirmed conclusion, since current magnitude is also influenced by fault resistance and the randomized load/source conditions, and may be partly acting as a fault-severity proxy rather than a pure distance proxy.

7. Results Summary

Task	Metric	Result
Fault Detection	Accuracy (Random Forest)	100%
Fault Type Classification	Overall Accuracy	96.0% (Random Forest), 95.6% (XGBoost), 80.9% (SVM)
Fault Type Classification	Accuracy, high-R band (30–100 Ω)	90.9%
Fault Location Estimation	MAE (overall)	9.82 km (6.54% of line length)
Fault Location Estimation	MAE, high-R band (30–100 Ω)	18.62 km (12.4% of line length)
Generalization	Holdout set location MAE (unseen data)	11.50 km

7.1 Key Findings

- The ML classifier's only systematic weakness is distinguishing LLL from LLLG faults — a physically explainable limitation shared by any method relying on zero-sequence content, since balanced three-phase faults barely produce it regardless of ground involvement.
- SVM underperforms specifically on the same LLL/LLLG pair (43–44% F1) while matching tree-based models elsewhere, suggesting its smooth RBF decision boundary is less suited to this particular overlapping-class distinction than tree-based partitioning.
- Classification is substantially more robust to fault resistance than location estimation (7-point accuracy drop vs. nearly tripled location error from low to high resistance).
- Unlike the conventional distance relay (which failed outright above 50 Ω in the companion project), the ML models degrade gradually across the full 0–100 Ω range tested, remaining usable even at high fault resistance.
- Performance on a genuinely unseen holdout batch (independent random seed) was consistent with the main test results, supporting that the models learned real underlying patterns rather than overfitting.

7.2 Limitations & Future Work

- Dataset is entirely simulation-based; real transmission line recordings (with measurement noise, CT/PT errors, and unmodeled system dynamics) would be needed to confirm real-world performance.
- Location regression accuracy at high fault resistance (18.6 km MAE) remains a meaningful limitation; further feature engineering or a dedicated high-resistance model could improve this.
- The current/location relationship observed in feature importance (Section 6.3) warrants further investigation to separate distance effects from resistance/load effects.
- Systematic sampling (e.g., Latin Hypercube Sampling) instead of pure random sampling could offer more even coverage of the fault-resistance / location parameter space for a future iteration.

- A neural network (e.g., 1D-CNN on raw waveforms) could be explored as an alternative to hand-crafted features, particularly to improve the LLL/LLLG distinction and high-resistance location accuracy.

8. Conclusion

This project built a complete, end-to-end pipeline — from physics-based Simulink simulation through automated data generation, feature engineering, and multi-model machine learning evaluation — for transmission line fault detection, classification, and location. The resulting ML models achieved strong, physically explainable performance, and directly demonstrated a meaningful advantage over the conventional distance relay approach specifically in the high-fault-resistance regime where impedance-based relays are known to fail. The project's greatest technical value lies not merely in the accuracy numbers achieved, but in the systematic validation performed at every stage — automated simulation validation, statistical sanity-checking of extracted features (which caught a real bug), cross-checking against known relay-protection theory, and testing on genuinely unseen data — reflecting a complete and rigorous engineering workflow rather than a single trained model in isolation.